Project title

StreamOps Agent — AI Assistant for Streaming Technical Product Managers

Project one-line summary

Build a MERN-stack prototype that connects streaming telemetry and docs to an AI agent which (1) detects anomalies, (2) drafts incident post-mortems and Jira/GitHub tickets with acceptance criteria, (3) drafts API contract templates from natural language requirements, and (4) proposes prioritized backlog items and A/B experiment designs — all surfaced through a chat UI and a simple scheduled automation.

Why this proves TPM competence

This project demonstrates end-to-end product + tech fluency: telemetry ingestion, observability, incident workflows, API design, partner integration thinking, and experiment design — plus the ability to translate operational data into product decisions via an LLM-powered agent.

Agent responsibilities (what the agent will *do*)

1. **Telemetry monitor & anomaly summarizer** — ingest simulated streaming KPIs and notify when thresholds/tails spike; generate concise human-readable incident summaries.
2. **Incident runbook & post-mortem draft generator** — given timeline & logs, produce a post-mortem draft with RCA, impact, and recommended follow-ups.
3. **Ticket/issue generator** — convert incidents or product requests into ready-to-triage GitHub issues (title, description, acceptance criteria, priority, labels).
4. **API contract generator** — from a short natural-language feature description, produce an OpenAPI skeleton plus suggested fields and example payloads.
5. **Experiment designer** — propose A/B experiment hypothesis, sample size, metrics to track, and rollout strategy based on current KPI baselines.
6. **Context retrieval (RAG)** — answer questions by retrieving context from runbooks, logs, and API docs using a local vector store (so the agent can cite supporting documents).

Tech stack (recommended) — free-first options and paid alternatives

- Backend: Node.js + Express (MERN stack). **Free**. LangChain will be used for orchestrating prompts/agents (LangChain JS).
- Database: MongoDB Atlas free tier for telemetry / events. **Free**.
- Vector store (RAG): Chroma (local or hosted) — **free/self-host** option available. **Free**.
- LLM provider: two options:
  - **Option A (paid but simplest):** OpenAI GPT-4 or GPT-4-style models (best quality, paid).
  - **Option B (low-cost / free tiers):** Hugging Face hosted models or a local open models (e.g., Llama-based via Hugging Face or local) — **lower cost / free tiers** but may need more infra. Hugging Face
- Frontend: React (create-react-app or Vite). **Free**.
- Auth/Devops: GitHub for code; deploy front on Vercel and backend on Render free tiers (or use Render for both). **Free** tiers available.
- Local media tooling (if simulating manifests): ffmpeg (open source). **Free**.
- Optional: Slack / GitHub integrations (webhooks) — **free** tiers.

Which elements are paid vs free

- **Free:** Node/React, MongoDB Atlas free tier, Chroma local, LangChain JS, Vercel/Render free tiers, ffmpeg, GitHub free, Hugging Face free model access (limited).
- **Paid:** OpenAI GPT-4 usage, hosted vector DB managed services, heavy cloud compute for large open models. You can complete a convincing prototype without paying by using smaller models or Hugging Face hosted trial tokens.

Deliverables (what you'll hand in)

1. GitHub repo with backend + frontend + infra (Dockerfile).

2. Deployed demo (Vercel/Render) showing chat UI + dashboard + example runbook/ticket flows.
3. Post-mortem & runbook templates generated by agent for at least two simulated incidents.
4. OpenAPI skeletons generated by the agent and a Postman collection.
5. Short 5–7 minute walkthrough video demonstrating key flows.
6. One-page impact report + 2–3 resume bullets with measured metrics from your simulations.

Architecture (high level)

- Telemetry ingestion endpoint → MongoDB raw events.
- Periodic aggregator job (Node worker) → computes KPIs (startup time median, p95 rebuffer, CDN error rate) and stores KPI docs in MongoDB.
- Vector store indexing: runbook docs + sample logs are embedded and pushed to Chroma.
- Agent layer (LangChain JS orchestration) calls LLM (OpenAI or HF) + retrieval from Chroma, composes outputs (incidents, tickets, OpenAPI).
- Task queue (simple in-process or BullMQ with Redis optional) to run scheduled anomaly checks and agent jobs.
- Frontend: React dashboard (KPI charts) + chat UI to ask the agent questions and accept generated issues.

Example repo layout

```
streamops-agent/
├─ backend/
│  ├─ src/
│  │  ├─ app.js (Express)
│  │  ├─ telemetryController.js
│  │  ├─ kpiWorker.js
│  │  ├─ agentController.js
│  │  └─ openapiGenerator.js
│  ├─ embeddings/
│  ├─ Dockerfile
```

```
│   └─ package.json
├─ frontend/
│   ├─ src/
│   │   ├─ App.jsx
│   │   ├─ Dashboard.jsx
│   │   └─ ChatAgent.jsx
│   └─ package.json
├─ infra/
│   ├─ docker-compose.yml
│   └─ README-infra.md
├─ docs/
│   ├─ sample_runbooks/
│   └─ README.md
└─ README.md
```

Sample agent prompts (starter)

Use these inside LangChain prompt templates or the chat UI.

1. **Anomaly summary**
   `You are StreamOps Agent. Given these KPI values for the last 10 minutes: {kpis}, explain whether there's an incident, estimate severity (low/medium/high), and provide 3 likely causes and 2 immediate mitigations. Use no more than 5 sentences.`
2. **Post-mortem draft**
   `Given this timeline and these logs: {timeline, logs}, produce a concise post-mortem with sections: Summary, Impact, Timeline, Root Cause, Remediation, Owners & ETA, and Suggestion for changes to SLA/monitoring. Keep it under 400 words.`
3. **Ticket generator**
   `Create a GitHub issue to resolve the top priority item from this post-mortem. Include title, description, acceptance criteria, test cases, priority, and suggested labels.`
4. **API contract**
   `From this request: "Allow partners to upload metadata and media reference for scheduled shows", produce an OpenAPI 3.0 skeleton for a POST /partner/ingest endpoint including authentication, request body schema, responses and example payload.`
5. **A/B experiment designer**
   `Given baseline rebuffer p95 = {value} and current retention conversion = {value}, propose an A/B test to test a new ABR ladder, include`

```
hypothesis, metric to measure, estimated sample size (approx), and
rollout plan.
```

Evaluation / acceptance criteria (how you'll measure success)

- Agent successfully generates a readable post-mortem from a simulated incident (qualitative pass).
- Agent produces an OpenAPI skeleton that can be loaded into Swagger UI without syntax errors (technical pass).
- Dashboard displays KPIs and triggers the agent when a simulated anomaly crosses thresholds (operational pass).
- Generated GitHub issue created via API (or stubbed locally) with acceptance criteria present (integration pass).
- Demo video + README explain tech tradeoffs and next steps.

12-week schedule (96 hours total: Weeks 1–12, Mon–Thu 2 hrs/day = 8 hrs/week)

Weeks 1–2 (16 hrs) — Foundations & scaffolding

- Day 1–2: Project plan, repo init, pick LLM provider (OpenAI or HF), set up GitHub.
- Day 3–4: Scaffold backend Express server, MongoDB Atlas free cluster, telemetry POST endpoint.
- Day 5–8: Scaffold React frontend and basic dashboard wireframe.

Weeks 3–4 (16 hrs) — Telemetry & KPI pipeline

- Day 9–12: Implement telemetry ingestion and store raw events.
- Day 13–16: Build KPI aggregator worker (compute median, p95, error rates) and API endpoints to fetch KPIs; connect dashboard charts.

Weeks 5–6 (16 hrs) — RAG & embeddings

- Day 17–20: Add sample docs/runbooks and logs; set up Chroma local vector store and embeddings pipeline (use HF/OpenAI embeddings).
- Day 21–24: Integrate retrieval logic and a simple “context fetch” endpoint.

Weeks 7–8 (16 hrs) — Agent basics: prompts & orchestration

- Day 25–28: Integrate LangChain JS, set up agent controller; wire sample prompts for anomaly summary and post-mortem.
- Day 29–32: Create agent endpoint and basic chat UI integration; test with sample incidents.

Weeks 9–10 (16 hrs) — Issue & API generator, automation

- Day 33–36: Implement ticket generator (issue payload), GitHub API stub (or real via token), and OpenAPI generator module.
- Day 37–40: Implement scheduled anomaly checker job to trigger agent automatically; add webhook simulation for alerts.

Weeks 11–12 (16 hrs) — Polish, deploy, document, demo

- Day 41–44: Polish UI, fix edge cases, write README and runbook templates.
- Day 45–48: Deploy backend & frontend to Render/Vercel, record 5–7 minute demo video, write one-page impact report and resume bullets.

Example metrics & resume bullets you can claim (if you implement as described)

- "Built StreamOps Agent — an LLM-powered assistant that ingests simulated streaming telemetry and auto-generates incident post-mortems and triage tickets; prototype reduced simulated mean time to triage by 40% in tests."
- "Authored API contract generator and Postman collection for partner ingest workflows—produced OpenAPI skeletons from natural language requirements in under 30 seconds per spec."

(Only claim measured values you actually implement — e.g., 'reduced mean time to triage by 40% in simulated scenario' — and be ready to demo.)

Minimal viable implementation to show in interviews (priority)

If you run low on time, prioritize these MVE parts:

1. Telemetry ingestion + KPI dashboard.

2. Agent that drafts post-mortems and a GitHub issue from a provided incident (connect to OpenAI or HF).
3. README + demo video.

That set demonstrates the most interview-relevant TPM competencies: monitoring, incident handling, and translating incidents into engineering work.

Security & privacy notes

- Don't send real customer logs or PII to hosted LLMs without redaction. If you must use real logs, run a local or private model or anonymize data first.
- Store API keys in environment variables and never commit them to Git.

Optional certifications that strengthen this project on your résumé

- Experimentation / A/B testing course (Optimizely Coursera) — optional.
- Cloud fundamentals (AWS Cloud Practitioner) — helps explain infra tradeoffs.
- Short LLM / prompt engineering course (various online micro-certs).

Next steps I can do for you (choose any)

1. Generate the exact starter repo skeleton (package.json, sample Express endpoints, sample React UI files).
2. Produce a ready-to-run Docker Compose that runs backend + local Chroma + Mongo for local dev.
3. Write the exact LangChain JS prompt templates and the sample code to call either OpenAI or a Hugging Face model.
4. Produce the 5–7 minute demo script and a short screen-capture checklist so you can record the walkthrough.

Which of the next steps would you like first? I recommend step **1 (starter repo skeleton)** so you can begin Week 14 immediately.

API contract generator

Explain this in plain english

Sure — here's a plain-English explanation of **API contract generator**, with what it does, why it matters, and how you'd show it off.

What it is (simple)

An **API contract generator** is a tool that turns a plain-English description of what a partner or product wants an API to do into a formal, machine-readable specification (an "API contract"). That spec describes the endpoint URLs, what data to send and receive, authentication, and example requests/responses — the exact instructions engineers use to build and test the API.

Why that matters

Instead of arguing over vague wording in Slack, the generator gives engineers, testers, and partners one clear source of truth they can run against tools (Swagger, Postman, code generators). It speeds up integration, reduces mistakes, and makes it obvious how a feature should behave.

What the generator actually does (step-by-step)

1. You type (or the agent reads) a short requirement in plain English: e.g., "Partners must be able to submit a show's metadata and a link to the video file."
2. The generator extracts the important pieces: endpoint name, HTTP verb (POST), fields (title, start_time, asset_url), required vs optional fields, auth method, and success/error responses.
3. It produces an **OpenAPI** file (YAML or JSON) and example requests/responses that can be loaded into tools like Swagger UI or Postman.
4. It can also spit out a ready-to-use Postman collection or a starter server/client code stub.

Inputs & outputs (concrete)

- Input: one or two sentences like "Create an endpoint for partners to upload show metadata and a reference URL, require OAuth token, return 201 on success."

- Output: an OpenAPI 3.0 document (or Postman collection) that defines `POST /partner/ingest`, the JSON schema for the request body, auth requirements, example `200/201` and `400/401` responses, and example curl commands.

What it proves for a Technical Product Manager

- You can translate product asks into technical specs — shows you can speak both product and engineering languages.
- You reduce ambiguity for engineers and partners (less rework).
- It demonstrates practical knowledge of REST, auth, response codes, and developer workflows — exactly the sort of technical fluency TPMs need.

Minimal acceptance criteria (how to know it works)

- The generated OpenAPI file loads cleanly in Swagger UI (no syntax errors).
- The spec includes request schema, response schema, auth details, and at least one example request.
- A developer can import the Postman collection or run a generated curl and see the right shape of request/response (you can stub the server if needed).

Quick example (one-line prompt → tiny OpenAPI snippet)

Prompt you'd give the agent:
`"Create an OpenAPI spec for POST /partner/ingest. Request body: partner_id (string), title (string), start_time (ISO8601), asset_url (string). Require Bearer token auth. Return 201 with {id, status} or 400 on validation error."`

Tiny OpenAPI output (shortened):

```
openapi: 3.0.0
paths:
  /partner/ingest:
    post:
      security:
        - bearerAuth: []
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
```

```
              required: [partner_id, title, asset_url]
              properties:
                partner_id: { type: string }
                title: { type: string }
                start_time: { type: string, format: date-time }
                asset_url: { type: string, format: uri }
      responses:
        '201':
          description: Created
          content:
            application/json:
              example: { id: "abc123", status: "queued" }
        '400':
          description: Validation error
```

Tools to use (free)

- Swagger Editor / Swagger UI — paste the OpenAPI YAML and preview.
- Postman (free) — import the generated collection.
- OpenAPI Generator (free CLI) — create server/client stubs.
- LLMs or LangChain to automate generation (OpenAI is paid; Hugging Face models / local models possible for low-cost).

How to present it in an interview

- Show the plain-English requirement, then show the generated OpenAPI and a Swagger UI screenshot.
- Explain the tradeoffs you encoded (what's required vs optional, error handling, rate-limit hints).
- Demonstrate one generated GitHub issue or acceptance criteria that you'd hand to an engineer, proving the link from product ask → technical task.

Short caution

Don't send real customer data or secrets to a public LLM. Redact logs before feeding them into any cloud model.

If you want, I can now:

- generate a small OpenAPI file from one of your real requirements, or

- produce the exact agent prompt templates that will reliably convert plain English into OpenAPI specs you can load into Swagger. Which would you like?

EDU_DOC_Resource_Pr...
Document

Please fill out this form using the information above.

Thought for...

EDU_DOC_Resource_Project+Design...

Project Design Document — StreamOps Agent (DRAFT)

Version: DRAFT
Date: 03/10/2026
Change log: initial draft

---

1. Project Title & Version Control

- **Project title:** StreamOps Agent — LLM-powered assistant for streaming TPM workflows.
- **Version & date:** DRAFT — 03/10/2026.
- **Change log:** Initial project definition and sprint plan.

2. Project Summary (2–3 sentences)

- Build a MERN-stack prototype that ingests simulated streaming telemetry and connects to a retrieval-augmented LLM agent which detects anomalies, drafts incident post-mortems, generates GitHub issues with acceptance criteria, and produces OpenAPI skeletons from plain-English requirements.
- This tool demonstrates the operational, integration, and product-decision tasks of a Technical Product Manager for streaming platforms and provides interview-ready artifacts (deployed demo, runbooks, and API specs).

3. Problem Statement / Use Case

- Streaming product teams lack a fast, repeatable way to turn operational signals and informal requests into triageable engineering work and clear API contracts; this slows incident response and partner integration.
- The target users are Technical Product Managers, SREs, and platform engineers who need quick incident summaries, reproducible runbooks, and machine-readable API contracts created from product requirements.

4. Goals and Objectives

- **Goal 1:** Implement a telemetry ingestion + KPI aggregation pipeline (startup time, p95 rebuffer, CDN errors) and surface these KPIs in a web dashboard.
- **Goal 2:** Build an agent (LangChain orchestration) that uses RAG (vector store) to draft incident post-mortems and convert plain-English requests into OpenAPI skeletons and GitHub issue payloads.
- **Goal 3:** Deliver deployable artifacts (GitHub repo, deployed demo, Postman/OpenAPI files, runbook templates, and a 5–7 minute demo video) within the 12-week course timeline.

5. Key Features / Functions

- Telemetry ingestion API to collect simulated streaming events and store raw logs.
- KPI aggregator job that computes median, p95, and error rates and exposes them via REST endpoints.
- React dashboard to visualize KPIs and surface anomaly alerts.
- Agent endpoints that: (a) summarize anomalies, (b) auto-draft post-mortems, (c) generate OpenAPI skeletons from text, and (d) create GitHub issue payloads.
- RAG capability (vector embeddings + Chroma) so the agent cites runbooks and logs when producing recommendations.
- Simple scheduler/automation to run anomaly checks and trigger agent workflows.

6. Tech Stack and Tools

- **Languages / frameworks:** Node.js (Express), React (Vite or CRA), JavaScript/TypeScript optional.
- **Datastore:** MongoDB Atlas free tier for events and KPI docs.
- **RAG / embeddings:** Chroma (local) + OpenAI or Hugging Face embeddings (OpenAI paid; HF/free options available).

- **Agent orchestration:** LangChain JS.
- **LLM provider:** OpenAI (GPT-4 — paid) or Hugging Face hosted/open models (low-cost/free).
- **Devops / hosting:** GitHub, Vercel (frontend), Render or Vercel serverless (backend), Docker for local dev.
- **Other tools:** ffmpeg (for any simulated manifest work), Postman/Swagger for API testing, Grafana or React charts for dashboards.

7. Architecture / Workflow (plain description)

- Clients (simulator or UI) POST streaming events → Express ingestion endpoint stores raw events in MongoDB.
- A scheduled KPI worker aggregates raw events into KPI documents (median, p95, error rates) and stores KPI snapshots.
- KPI snapshots are fetched by the React dashboard and by the anomaly checker; when thresholds or tail metrics breach rules, the anomaly checker queues an agent job.
- The agent (LangChain) retrieves contextual runbooks/log snippets from Chroma, calls an LLM (OpenAI/HF) with prompt templates, and returns outputs: a concise incident summary, a drafted post-mortem, a GitHub issue payload, or an OpenAPI skeleton; the UI lets users review and push issues to GitHub (or export OpenAPI files).

8. Timeline / Weekly Milestones (4-week sprint shown; this project spans 12 weeks)

> **Note:** The template below represents the initial 4-week sprint; the full project is scheduled across 12 weeks (see full schedule in project repo README).

- **Week 1 — Project Definition:** Finalize scope, select LLM provider (OpenAI vs HF), create repo, and scaffold backend + frontend skeletons.
- **Week 2 — Requirements and Goals:** Define KPI schema, acceptance criteria for agent outputs (post-mortem, OpenAPI), and required sample runbooks/log examples.
- **Week 3 — Solution Planning and Design:** Design data models, RAG indexing flow, agent prompt templates, and initial UI wireframes for dashboard + chat.

- **Week 4 — Review and Refinement:** Implement basic ingestion endpoint, run a simple KPI aggregator, test end-to-end telemetry → dashboard flow, and document sprint demo and risks.

(Full 12-week plan with week-by-week tasks and demos will be included in the repo README and course schedule; initial sprint focuses on foundations and a minimum viable demo.)

9. Risks and Risk Mitigation

- **Risk — LLM costs & rate limits:** Using commercial LLMs can be costly for repeated experiments. **Mitigation:** use small sample sizes in dev, prefer Hugging Face free models for prototyping, cache agent responses, and implement throttles.
- **Risk — Data privacy / PII in logs:** Sending raw logs to hosted LLMs may expose sensitive data. **Mitigation:** redact or synthesize logs before embedding; use local models when necessary.
- **Risk — Model hallucination / inaccurate specs:** LLM may produce incorrect OpenAPI or acceptance criteria. **Mitigation:** require human review, include unit validation (load generated OpenAPI into Swagger) and acceptance tests.
- **Risk — Infra complexity & time limits:** Setting up vector stores, transformers, and hosting can exceed the 96-hour constraint. **Mitigation:** scope the MVP tightly (use Chroma local, minimal dataset), prioritize telemetry + agent post-mortem flow first.

10. Evaluation Criteria

- **Functional correctness:** The agent must produce at least two valid post-mortem drafts from simulated incidents and one OpenAPI file that loads without syntax errors in Swagger UI.
- **Operational behavior:** Dashboard shows KPIs and triggers an anomaly alert that results in an agent-generated GitHub issue payload (or local stub).
- **Deliverables & reproducibility:** Public GitHub repo with README, Postman/OpenAPI artifacts, a deployed demo (or clear local run instructions), and a 5–7 minute demo video.
- **Interview readiness:** A one-page impact summary and 2–3 resume bullets that describe measurable results from the prototype (e.g., simulated reduction in mean time to triage).

11. Future Considerations

- For production parity: integrate real CDN logs/transcode metrics, add authenticated partner onboarding with OAuth, and replace simulated alerts with real SRE tooling (Datadog).
- Improve model reliability by adding verification layers (schema validators for OpenAPI, unit tests for generated acceptance criteria) and consider integrating LaunchDarkly or Unleash for feature flags and rollout.
- Long-term scaling: move embeddings into a managed vector DB, add CI/CD for model prompt changes, and implement role-based access for runbooks and incident outputs.

If you'd like, I will now: (A) generate the full 12-week week-by-week milestone table to attach to this document, or (B) create the starter repo skeleton (package.json, basic Express endpoints, and a minimal React UI) and add Docker Compose for local dev. Which should I produce next?

Sources

SREs,

What does this mean?

Great — here's a clear, plain-English explanation of **SREs** and what that label means in your project context, plus practical notes on how your **StreamOps Agent** should support them.

What "SREs" means (simple)

SREs = Site Reliability Engineers. They're engineers whose job is to make a service reliable, scalable, and operable in production — keeping video playback working, APIs responding, and outages short.

What SREs actually do (short bullets)

- **Run the system 24/7:** maintain on-call rotations and respond to alerts when something breaks.

- **Monitor & measure:** define and track SLIs/SLOs (e.g., startup time, rebuffer p95), build dashboards and alerts.
- **Incident response:** own triage, mitigation, and blameless post-mortems after outages.
- **Automation & tooling:** automate repetitive work (deployments, rollbacks, health checks) and build runbooks.
- **Capacity & performance:** plan for scale (CDN behavior, transcoder throughput), tune systems to meet latency and cost targets.
- **Reliability engineering:** add retries, circuit breakers, caching, and observability to reduce incident frequency.

Why SREs matter to a Technical Product Manager (TPM)

- SREs translate product/feature changes into operational reality — they surface what breaks, why it matters, and how to measure success.
- TPMs need SRE buy-in when prioritizing infra work (e.g., ABR tweaks, new ingest APIs) because SREs own uptime/latency SLAs.

How your StreamOps Agent should help SREs (practical)

- **Auto-generate a concise incident summary** (severity, timeline, top suspected cause) so SREs can triage faster.
- **Produce runbook steps and rollback instructions** that SREs can execute during on-call.
- **Create monitorable acceptance criteria** (exact metric thresholds and queries) the SRE team can turn into alerts.
- **Draft GitHub issues or tasks** with clear owner, priority, and acceptance tests so SREs don't have to guess next steps.
- **Embed logs & relevant context** (timestamps, request IDs, correlation IDs) and show which parts are redacted for privacy.

What SREs expect to see in a good incident ticket (use this template)

- **Title / severity:** short title + Sev (e.g., Sev2 — Playback rebuffer p95 spike)

- **Impact:** what users and business metrics are affected (e.g., 10% drop in watch time).
- **Timeline:** when it started, key events, and detection timestamp.
- **Observed metrics & links:** direct links or queries for dashboards (startup time p50/p95, rebuffer rate).
- **Repro steps / sample requests:** minimal reproducible case or curl command.
- **Logs & request IDs:** annotated snippets or pointers to full logs (redacted).
- **Immediate mitigation:** what to try now (rollback, scale CDN, purge cache).
- **Owner & ETA:** who's assigned and expected next checkpoint.
- **Post-mortem follow-ups:** short list of planned remediation and owners.

Quick tips for working well with SREs

- Use measurable language: "reduce rebuffer p95 from 8s to 3s" not "improve buffering."
- Provide test data and a reproducible curl or script; SREs love reproducible cases.
- Always include exact timestamps and request IDs — these make log searches fast.
- Avoid dumping raw production logs to public LLMs — redact or synthesize before using AI.
- Offer a rollback plan in every change request so SREs can assess risk quickly.

One-sentence summary you can use in interviews

"SREs (Site Reliability Engineers) are the team that keeps streaming services live and performant — they handle monitoring, on-call incident response, automation, and capacity planning — and your StreamOps Agent should give them concise incident summaries, runnable runbooks, and validated alert queries so they can detect, mitigate, and prevent outages faster."

If you want, I'll convert that incident ticket template into a one-click GitHub issue generator prompt the agent can use.

generate a workflow diagram based on the document 1-11.

Thought for 4 seco...

Workflow diagram — StreamOps Agent (based on Project Doc §§1–11)

Below is a clear workflow diagram (Mermaid) showing how the pieces from the Project Design Document (sections 1–11) connect and the runtime flow from telemetry → agent → human action.
Paste the Mermaid code into any Mermaid-enabled renderer (Obsidian, VS Code Mermaid Preview, Mermaid Live Editor, GitHub Markdown with mermaid support, etc.) to view the visual diagram.

What is this?

Plain-English step-by-step (textual workflow)

1. **Event generation:** A telemetry simulator or real clients POST streaming events (startup times, rebuffer events, CDN errors) to the ingestion API. (Doc §5, §6)
2. **Raw storage:** The ingestion API stores raw events/logs in MongoDB for later analysis. (Doc §6, §7)
3. **KPI aggregation:** A scheduled worker (cron) reads raw events and computes KPIs (median, p95, error rates) and stores KPI snapshots. (Doc §4, §8)

4. **Dashboard & alerts:** The React dashboard reads KPI snapshots and shows charts; simple alerting rules surface anomalies to users. (Doc §2, §5)
5. **Anomaly detection:** A scheduled anomaly checker inspects KPIs; when thresholds or tail metrics breach, it enqueues an agent job. (Doc §4, §10)
6. **RAG context:** The agent controller queries the vector store (Chroma) which contains indexed runbooks, docs, and redacted logs to provide context. (Doc §5, §7)
7. **LLM + agent orchestration:** The agent (LangChain JS) calls the LLM provider (OpenAI or Hugging Face) plus retrieved context to generate structured outputs: post-mortems, GitHub issue payloads, OpenAPI skeletons, A/B experiment plans, and runbooks. (Doc §5, §6)
8. **Human review & action:** TPMs and SREs review the agent outputs, refine them if needed, then push approved tickets to GitHub, notify stakeholders, or execute mitigation/runbook steps. (Doc §9, §10)
9. **Engineering work & rollout:** Engineering implements fixes or features, deploys changes, and the cycle repeats with telemetry reflecting the outcome. (Doc §4, §11)

Mapping diagram nodes to Project Document sections (1–11)

- **Project title & summary (§1–2):** Top-level goal is the StreamOps Agent; diagram shows the whole system.
- **Problem statement & goals (§3–4):** Anomaly detection, faster triage, and generating machine-readable specs (post-mortems, OpenAPI) are central workflows.
- **Key features (§5):** Ingestion, KPI aggregation, dashboard, agent outputs (post-mortem, OpenAPI, tickets).
- **Tech stack (§6):** Nodes labeled with Express, MongoDB, Chroma, LangChain, LLM.
- **Architecture/workflow (§7):** The flow from client → ingestion → storage → aggregator → agent → human → engineering is the architecture.
- **Timeline (§8):** Scheduler and workers represent recurring week-by-week work; the diagram supports sprinted delivery.
- **Risks (§9):** RAG + LLM nodes emphasize redaction and verification (mitigations: human review).

- **Evaluation (§10):** Outputs (valid OpenAPI, valid post-mortem, working dashboard) are the measurable endpoints.
- **Future considerations (§11):** Integrations (real CDN logs, Datadog) and managed services are optional next steps off the diagram.

If you want this as a PNG/SVG

I can export the diagram into a downloadable SVG/PNG file (or generate a ready-to-paste Mermaid file). Tell me which format you prefer (SVG or PNG) and I'll produce the file and the exact Mermaid text ready for your editor.

Would you like the diagram exported as an SVG now, or do you want it adjusted (add more detail, show sequence numbers, or show deployment zones like "local dev / cloud / managed services")?

Generate mermaid code to create a high level diagram of the following steps: Event generation: A telemetry simulator or real clients POST streaming events (startup times, rebuffer events, CDN errors) to the ingestion API. (Doc §5, §6)

Raw storage: The ingestion API stores raw events/logs in MongoDB for later analysis. (Doc §6, §7)

KPI aggregation: A scheduled worker (cron) reads raw events and computes KPIs (median, p95, error rates) and stores KPI snapshots. (Doc §4, §8)

Dashboard & alerts: The React dashboard reads KPI snapshots and shows charts; simple alerting rules surface anomalies to users. (Doc §2, §5)

Anomaly detection: A scheduled anomaly checker inspects KPIs; when thresholds or tail metrics breach, it enqueues an agent job. (Doc §4, §10)

RAG context: The agent controller queries the vector store (Chroma) which contains indexed runbooks, docs, and redacted logs to provide context. (Doc §5, §7)

LLM + agent orchestration: The agent (LangChain JS) calls the LLM provider (OpenAI or Hugging Face) plus retrieved context to generate structured outputs: post-mortems, GitHub issue payloads, OpenAPI skeletons, A/B experiment plans, and runbooks. (Doc §5, §6)

Human review & action: TPMs and SREs review the agent outputs, refine them if needed, then push approved tickets to GitHub, notify stakeholders, or execute mitigation/runbook steps. (Doc §9, §10)

Engineering work & rollout: Engineering implements fixes or features, deploys changes, and the cycle repeats with telemetry reflecting the outcome. (Doc §4, §11)

Here's the Mermaid code — paste this into any Mermaid-enabled renderer (Mermaid Live Editor, VS Code, Obsidian, GitHub with mermaid support) to view the high-level workflow diagram.

```
flowchart LR
  %% Sources
  A[Telemetry Simulator / Clients<br/>POST events<br/>(startup, rebuffer,

  %% Ingestion & Storage
  B[Ingestion API<br/>(Express) ]
  C[Raw Events / Logs<br/>(MongoDB)]

  %% Aggregation & KPIs
  D[KPI Aggregator Worker<br/>(cron) ]
  E[KPI Snapshots<br/>(MongoDB)]

  %% Dashboard & Alerts
  F[Dashboard (React) + Alerts<br/>KPI charts & rules]

  %% Anomaly -> Agent
  G[Anomaly Checker<br/>(scheduled) ]
  H[Agent Job Queue]

  %% RAG & Agent
```

```
I[Vector Store (Chroma)<br/>Indexed runbooks/docs/logs]
J[Agent Controller<br/>(LangChain JS)]
K[LLM Provider<br/>(OpenAI / Hugging Face)]

%% Outputs
O1[Post-mortem Drafts]
O2[GitHub Issue Payloads]
O3[OpenAPI Skeletons]
O4[A/B Experiment Plans]
O5[Runbooks / Mitigations]

%% Human review & actions
R[Human Review<br/>(TPM / SRE)]
S[Create Issue / Notify Stakeholders / Execute Runbook]
T[Engineering Implementation & Rollout]
U[Production Systems (changes reflected in telemetry)]

%% Flow edges
A -->|POST events| B
B -->|store logs| C
C -->|read raw events| D
D -->|compute KPIs| E
E -->|read KPIs| F
E -->|read KPIs| G
G -->|enqueue agent job when threshold breached| H
H -->|job| J
J -->|retrieve context| I
J -->|call LLM + context| K
K -->|structured outputs| J
J -->|produce| O1 & O2 & O3 & O4 & O5
O1 & O2 & O3 & O4 & O5 --> R
R -->|approve/modify| S
S -->|create tasks / push to backlog| T
T -->|deploy fixes/features| U
U -->|telemetry reflects change| A

%% grouping (visual)
classDef dataLayer fill:#f3f7ff,stroke:#1f6feb;
classDef agentLayer fill:#fff7e6,stroke:#ff8c00;
classDef humanLayer fill:#e8fff1,stroke:#0f9d58;

class B,C,D,E,F,G dataLayer;
class I,J,K agentLayer;
class R,S,T humanLayer;
```